《2026Fall 机器学习与数据挖掘》作业1实验报告

基于不同损失函数与核函数的手写数字二分类实验

课程名称： 机器学习与数据挖掘
实验名称： 手写数字 8 与 9 的二分类
姓名： 唐咏妍
学号： 24325241
实验完成日期： 2026年11月3日

一、实验内容

本实验以老师提供的手写数字图像数据集为研究对象，选取数字 8 和数字 9 两个类别，完成一个二分类任务。

实验主要围绕支持向量机（SVM）及不同损失函数下的线性分类模型展开，采用调用机器学习库与手动编写算法相结合的方式，实现并比较以下四种模型：

1. **Linear SVM：** 使用线性核函数的支持向量机分类模型。
2. **RBF SVM：** 使用径向基函数（RBF）核的支持向量机分类模型。
3. **Hinge Loss Classifier：** 基于合页损失函数（Hinge Loss）和梯度下降法手动实现的线性分类模型。
4. **Cross-Entropy Classifier：** 基于交叉熵损失函数（Cross-Entropy）和梯度下降法手动实现的逻辑回归模型。

具体实验任务包括：

- 对原始图像数据进行加载、预处理和数据集划分。
- 使用 Scikit-learn 实现线性核 SVM 和 RBF 核 SVM，并进行超参数搜索。
- 手动实现 Hinge Loss 分类器，包括损失函数、梯度计算及梯度下降训练过程。
- 手动实现 Cross-Entropy 分类器，包括 Sigmoid 函数、交叉熵损失、梯度计算及梯度下降训练过程。
- 研究学习率和正则化系数对模型性能的影响。
- 通过损失曲线、错分方向统计及分类评价指标，对不同模型进行比较和分析。

二、实验目的

1. 理解监督学习中的二分类问题，掌握手写数字图像分类的基本流程。
2. 理解支持向量机的基本原理，掌握线性核与 RBF 核的区别及其应用。
3. 理解 Hinge Loss 和 Cross-Entropy 两种损失函数的数学定义、优化目标及其差异。
4. 掌握梯度下降算法的基本思想，能够独立推导并编写损失函数的梯度计算代码。
5. 理解学习率和正则化系数对模型收敛速度、分类性能及泛化能力的影响。
6. 掌握 Accuracy、Precision、Recall、F1-score 和混淆矩阵等分类评价方法。
7. 通过不同模型的实验对比，加深对损失函数、核函数及模型复杂度的理解。

三、实验原理

3.1 二分类问题的数学表示

本实验的目标是根据一张手写数字图像判断其属于数字 8 还是数字 9。原始图像大小为 28×28，经过展平后，每个样本表示为一个 784 维特征向量：

$$x_i \in \mathbb{R}^{784}$$

像素值进一步由 $[0, 255]$ 归一化到 $[0, 1]$。归一化能在不改变图像中笔画相对结构的同时减小特征数值尺度，使梯度下降时各维特征对参数更新的影响更稳定，也避免较大的像素值造成梯度过大。

四种模型虽然完成的是同一个二分类任务，但对标签的数学编码并不完全相同。SVM 和 Hinge Loss 模型需要利用样本的符号计算分类间隔，因此采用：

$$y_i = \begin{cases} +1, & \text{数字为 } 9 \\ -1, & \text{数字为 } 8 \end{cases}$$

Cross-Entropy 模型将输出解释为“样本是数字 9 的概率”，因此采用：

$$y_i = \begin{cases} 1, & \text{数字为 } 9 \\ 0, & \text{数字为 } 8 \end{cases}$$

对于线性模型，首先计算样本的判别得分：

$$f(x_i) = w^T x_i + b$$

其中，$w \in \mathbb{R}^{784}$ 为权重向量，决定各像素特征对分类结果的影响；b 为偏置项，用于平移分类边界。若直接依据得分符号分类，则 $f(x_i) = 0$ 构成特征空间中的决策超平面，$f(x_i) \geq 0$ 判为数字 9，$f(x_i) < 0$ 判为数字 8。模型训练的实质，就是根据训练样本不断调整 w 和 b，使两类样本位于决策边界的不同侧，并尽量提高模型在未见样本上的分类能力。

3.2 支持向量机的一般理论

3.2.1 最大间隔思想

支持向量机（Support Vector Machine，SVM）不仅要求训练样本被正确分类，还希望分类边界与两类中距离边界最近的样本之间具有尽可能大的间隔。对采用 $y_i \in \{-1, +1\}$ 编码的样本，$y_i f(x_i) > 0$ 表示分类正确，而 $y_i f(x_i)$ 的数值越大，表示样本在正确一侧且离决策边界越远。将函数间隔规范为 1 后，线性可分条件写为：

$$y_i(w^T x_i + b) \geq 1, \quad i = 1, 2, \ldots, N$$

两条间隔边界分别为 $w^T x + b = 1$ 和 $w^T x + b = -1$，它们之间的几何距离为：

$$\frac{2}{\|w\|}$$

因此，最大化间隔等价于最小化 $\frac{1}{2}\|w\|^2$。理想线性可分情况下的硬间隔 SVM 可表示为：

$$\min_{w,b} \frac{1}{2}\|w\|^2$$

$$\text{s.t.} \quad y_i(w^T x_i + b) \geq 1$$

实际手写图像中可能存在书写模糊、形态相似或噪声样本，若要求所有样本都严格满足间隔约束，模型容易受到少量异常样本影响。因此，软间隔 SVM 引入松弛变量 $\xi_i \geq 0$：

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}\xi_i$$

$$\text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i$$

这里的 $C > 0$ 是误差惩罚系数。C 较大时，模型更重视减少训练样本的间隔违背和误分类，可能形成较复杂的边界；C 较小时，模型允许更多样本落入间隔内，以换取更宽的间隔和更强的正则化。由此可见，C 控制的是“训练误差”与“模型复杂度”之间的权衡，并非越大越好。

3.2.2 支持向量与决策函数

距离决策边界最近、对最优超平面位置起关键作用的样本称为支持向量。远离边界且已经被稳定正确分类的样本，即使发生小幅移动，通常也不会改变最终边界；支持向量的变化则可能直接改变分类面。这也是 SVM 名称的来源。

从对偶问题的角度，SVM 的决策函数可以写为：

$$f(x) = \sum_{i=1}^{N}\alpha_i y_i K(x_i, x) + b$$

其中，α_i 是对偶变量，只有支持向量对应的 α_i 通常非零；$K(x_i, x)$ 为核函数。最终预测由得分符号确定：

$$\hat{y} = \text{sign}(f(x))$$

本实验使用 Scikit-learn 的 SVC 完成 SVM 训练，并在验证集上以 F1-score 为依据选择超参数。这样可以避免直接使用测试集调参，使测试结果更能反映模型的泛化性能。

3.3 核函数与非线性分类

3.3.1 线性核

线性核定义为：

$$K(x_i, x_j) = x_i^T x_j$$

使用线性核时，模型在原始 784 维像素空间中寻找线性超平面。其优点是结构简单、训练和预测相对高效，且权重可以理解为不同像素位置对分类的贡献；其局限是只能形成线性决策边界。如果数字 8 与数字 9 的差异不能由所有像素的一个线性组合充分表达，线性核的分类能力会受到限制。

3.3.2 核技巧与 RBF 核

对于线性不可分数据，可以设想通过非线性映射 $\phi(x)$ 将样本变换到更高维特征空间，再在该空间中建立线性分类器。然而，直接计算高维映射可能带来很大的计算和存储开销。核技巧利用：

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

直接计算样本映射后的内积，而不显式构造 $\phi(x)$。因此，模型仍可使用线性分类器的优化框架，却能够在原始输入空间中得到非线性决策边界。

本实验采用的径向基函数（Radial Basis Function，RBF）核为：

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

RBF 核以样本之间的欧氏距离衡量相似程度。两个样本越接近，$K(x_i, x_j)$ 越接近 1；距离越远，核函数值越接近 0。参数 γ 决定单个训练样本的影响范围：

- 当 γ 较小时，核函数随距离衰减较慢，单个样本的影响范围较广，得到的决策边界通常较平滑，但过小可能导致欠拟合。
- 当 γ 较大时，单个样本的影响集中在较小邻域内，模型可以形成更曲折的边界，但过大可能使模型过度贴合训练样本并产生过拟合。

因此，RBF SVM 的 C 与 γ 需要联合考虑：C 决定对间隔违背的容忍程度，γ 决定边界的局部变化尺度。本实验通过遍历候选参数组合并比较验证集 F1-score，选择二者的合适搭配。

3.4 Hinge Loss 线性分类模型及梯度推导

3.4.1 Hinge Loss 与软间隔 SVM 的关系

软间隔约束中的最小松弛量可以写为：

$$\xi_i = \max\left(0, 1 - y_i(w^T x_i + b)\right)$$

将其代回软间隔 SVM 的目标函数，即可得到由正则化项和 Hinge Loss 组成的无约束优化形式。为与代码中的“平均样本损失”保持一致，本实验手动实现的目标函数为：

$$L_{\text{hinge}}(w, b) = \frac{1}{N} \sum_{i=1}^{N} \max\left(0, 1 - y_i(w^T x_i + b)\right) + \frac{\lambda}{2}\|w\|^2$$

其中，N 为样本数，λ 为 L2 正则化系数。该目标与软间隔 SVM 都体现了“扩大分类间隔并惩罚间隔违背”的思想，但由于损失的缩放方式和优化实现不同，代码中的 λ 不能简单地与 SVC 中的 C 视为数值上严格互为倒数。

令样本的带符号间隔为：

$$m_i = y_i(w^T x_i + b)$$

根据 m_i 的取值，可以分三种情况理解 Hinge Loss：

1. 当 $m_i \geq 1$ 时，样本分类正确且位于间隔边界之外，样本损失为 0。
2. 当 $0 < m_i < 1$ 时，样本分类正确但离决策边界过近，仍会受到惩罚。
3. 当 $m_i \leq 0$ 时，样本被错误分类，且离正确一侧越远，损失越大。

因此，Hinge Loss 不只关心“是否分对”，还要求正确样本与边界保持足够距离。训练过程中，$m_i \geq 1$ 的样本不参与经验损失部分的梯度计算，参数更新主要由间隔内样本和误分类样本推动，这与支持向量决定分类边界的思想一致。

3.4.2 Hinge Loss 的梯度推导

对单个样本，记：

$$\ell_i = \max(0, 1 - m_i)$$

当 $m_i < 1$ 时，$\ell_i = 1 - y_i(w^T x_i + b)$，分别对 w 和 b 求导可得：

$$\frac{\partial \ell_i}{\partial w} = -y_i x_i, \qquad \frac{\partial \ell_i}{\partial b} = -y_i$$

当 $m_i > 1$ 时，$\ell_i = 0$，其梯度为 0。在不可导点 $m_i = 1$ 处可选取一个次梯度；现有代码通过判断 `margin < 1`，将等于 1 的样本归入梯度为 0 的一侧。定义指示函数：

$$I_i = \begin{cases} 1, & m_i < 1 \\ 0, & m_i \geq 1 \end{cases}$$

再考虑 $\frac{\lambda}{2}\|w\|^2$ 对 w 的导数为 λw，得到完整梯度：

$$\nabla_w L_{\text{hinge}} = -\frac{1}{N}\sum_{i=1}^{N} I_i y_i x_i + \lambda w$$

$$\nabla_b L_{\text{hinge}} = -\frac{1}{N}\sum_{i=1}^{N} I_i y_i$$

正则化项中不包含 b，因此偏置梯度没有 λb。代码先通过布尔掩码选出 $m_i < 1$ 的“有效样本”，再利用矩阵乘法一次性计算上述梯度，这与逐样本求和在数学上等价，但计算效率更高。

3.5 Cross-Entropy 线性分类模型及梯度推导

3.5.1 概率建模与交叉熵损失

Cross-Entropy 分类器采用逻辑回归形式。模型先计算线性得分：

$$z_i = w^T x_i + b$$

再通过 Sigmoid 函数将任意实数映射到 $(0, 1)$：

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

其中，p_i 被解释为样本属于数字 9 的预测概率，即 $P(y_i = 1 \mid x_i)$；$1 - p_i$ 则表示属于数字 8 的预测概率。由于 $\sigma(0) = 0.5$，以 0.5 为概率阈值等价于以 $z_i = 0$ 为决策边界：

$$\hat{y}_i = \begin{cases} 1, & p_i \geq 0.5 \\ 0, & p_i < 0.5 \end{cases}$$

在二分类问题中，单个样本标签服从伯努利分布，其似然可写为 $p_i^{y_i}(1 - p_i)^{1 - y_i}$。对全体样本的似然取负对数并求平均，就得到二元交叉熵；再加入 L2 正则化后，本实验的目标函数为：

$$L_{\text{CE}}(w, b) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right] + \frac{\lambda}{2}\|w\|^2$$

该损失会对“高置信度但预测错误”的样本给予很大惩罚。例如，当真实标签 $y_i = 1$ 而 p_i 接近 0 时，$-\log p_i$ 会迅速增大。代码在计算对数前先令 $\varepsilon = \texttt{np.finfo(p.dtype).eps}$，再将概率截断到 $[\varepsilon, 1 - \varepsilon]$。对于本实验的 `float32` 概率数组，这避免了固定极小量在计算 $1 - \varepsilon$ 时被舍入为 1、进而出现 $\log(0)$ 的问题；截断只服务于损失计算，不改变 0.5 的预测阈值。

3.5.2 Cross-Entropy 的梯度推导

为了说明梯度为何可化为简洁形式，先考虑单个样本的交叉熵：

$$\ell_i = -\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]$$

Sigmoid 函数的导数为：

$$\frac{\partial p_i}{\partial z_i} = p_i(1-p_i)$$

交叉熵对概率的导数为：

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i}$$

根据链式法则，两式相乘并整理得：

$$\frac{\partial \ell_i}{\partial z_i} = p_i - y_i$$

又因为 $\frac{\partial z_i}{\partial w} = x_i$、$\frac{\partial z_i}{\partial b} = 1$，所以全体样本的梯度为：

$$\nabla_w L_{\text{CE}} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)x_i + \lambda w = \frac{1}{N}X^T(p-y) + \lambda w$$

$$\nabla_b L_{\text{CE}} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)$$

其中，$p - y$ 表示预测概率与真实标签之间的误差。当 $p_i > y_i$ 时，梯度更新会使对应得分下降；当 $p_i < y_i$ 时，更新会使得分上升。由此，模型逐步提高正确类别的预测概率。代码中的 `error = p - y`、`X.T @ error / len(X)` 和 `np.mean(error)` 分别对应上述误差向量、权重梯度和偏置梯度。

3.6 Hinge Loss 与 Cross-Entropy 的理论比较

两种手动实现的模型都使用线性函数 $w^Tx + b$ 建立决策边界，也都加入 L2 正则化并使用全批量梯度下降训练，但它们对模型输出的解释和对样本的惩罚方式不同。

比较维度	Hinge Loss	Cross-Entropy
标签编码	$\{-1, +1\}$	$\{0, 1\}$
输出含义	带符号的分类得分与间隔	属于正类的概率
优化重点	正确分类并使间隔至少达到 1	提高真实类别的预测概率
样本作用	间隔达到 1 后，经验损失梯度为 0	除非概率与标签完全一致，否则通常仍有梯度
损失光滑性	在间隔点不可导，使用次梯度	连续可导，便于梯度优化
概率解释	得分本身不是概率	可直接输出概率，但概率质量仍受模型拟合影响

从优化行为看，Hinge Loss 会把注意力集中在分类边界附近和被误分类的样本上，强调最大间隔；Cross-Entropy 则持续调整所有样本的预测概率，尤其强烈惩罚高置信度的错误预测。由于本实验的两种手动模型都

只学习线性边界，二者的差异主要来自损失函数，而 RBF SVM 的额外优势还来自非线性核映射。因此，对最终性能的比较应同时考虑损失函数、模型表示能力和超参数设置，不能仅由某一次指标高低断定某种损失函数在所有任务上都更优。

3.7 L2 正则化与梯度下降

3.7.1 L2 正则化

本实验在 Hinge Loss 和 Cross-Entropy 的经验损失之后加入：

$$L_{\text{reg}} = \frac{\lambda}{2}\|w\|^2$$

其梯度为 λw。在每次参数更新中，这一项会把权重向 0 的方向拉回，因此 L2 正则化也可理解为一种权重衰减。较小的权重通常对应更平滑、对输入扰动不那么敏感的决策函数，有助于降低过拟合风险。λ 太小可能无法充分约束模型，太大则会过度压缩权重，使模型难以拟合数据并产生欠拟合。偏置 b 不直接控制各特征方向上的复杂度，现有实现没有对其正则化。

3.7.2 全批量梯度下降

设完整目标函数记为 $L(w, b)$，每轮训练先在全部训练样本上计算平均梯度，再按以下规则更新：

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w L(w^{(t)}, b^{(t)})$$

$$b^{(t+1)} = b^{(t)} - \eta \nabla_b L(w^{(t)}, b^{(t)})$$

其中，t 表示迭代轮次，η 为学习率。负梯度方向是损失在当前位置下降最快的局部方向。学习率决定每次更新的步长：若过小，损失下降缓慢，有限轮数内可能尚未充分收敛；若过大，参数可能越过低损失区域，造成振荡甚至不收敛。因此，本实验分别比较多个学习率，并依据验证集 F1-score 选择合适值。

本实验使用全批量梯度下降，即每轮都使用全部训练样本。其梯度方向稳定、损失曲线通常较平滑，但单轮计算量随样本数增加。参数初始化为全零并不会像多层神经网络那样造成同层神经元的对称性问题，因为这里仅有一个线性输出单元；首轮梯度即可根据输入特征和标签产生非零更新。训练过程中同时记录训练损失与验证损失：若训练损失持续下降而验证损失上升，通常提示模型开始过拟合；若二者都较高，则可能存在欠拟合、学习率不合适或训练轮数不足等问题。

3.8 模型评价指标

本实验将数字 9 设为正类，数字 8 设为负类。混淆矩阵中的四种结果定义为：

- TP（True Positive）：真实为 9，预测也为 9。
- TN（True Negative）：真实为 8，预测也为 8。
- FP（False Positive）：真实为 8，却预测为 9。
- FN（False Negative）：真实为 9，却预测为 8。

混淆矩阵不仅给出错误总数，还能区分两种错误方向。基于这四个计数，可以得到以下指标。

（1）准确率 Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy 表示全部样本中预测正确的比例，能够反映总体分类性能。但当类别分布严重不平衡时，模型可能只偏向多数类也获得较高准确率，因此还需要结合其他指标判断。

（2）精确率 Precision

$$Precision = \frac{TP}{TP + FP}$$

Precision 回答“模型预测为数字 9 的样本中，有多少确实是 9”。较高的 Precision 表明将数字 8 错判为 9 的情况较少，即 FP 较少。

（3）召回率 Recall

$$Recall = \frac{TP}{TP + FN}$$

Recall 回答“所有真实数字 9 中，有多少被成功识别”。较高的 Recall 表明漏判数字 9 的情况较少，即 FN 较少。

（4）F1-score

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

F1-score 是 Precision 与 Recall 的调和平均值。只有两者都较高时，F1-score 才会较高，因此适合综合衡量模型对正类的识别质量。本实验在验证阶段使用 F1-score 选择超参数，并在独立测试集上同时报告 Accuracy、Precision、Recall、F1-score 以及两类错分方向的数量，以避免单一指标掩盖模型的具体错误类型。

四、实验环境与数据集

4.1 实验环境

项目	配置
操作系统	Windows 11
编程语言	Python
开发环境	VS Code / Python 虚拟环境
数值计算	NumPy
数据加载	PyTorch
机器学习库	Scikit-learn
数据可视化	Matplotlib

4.2 数据集介绍

本实验使用教师提供的手写数字 8 和 9 图像数据集。

每张图像的尺寸为 28×28，原始像素类型为 `torch.uint8`，像素范围为 [0,255]。

数据集具体情况如下：

数据集	样本数量	数字8	数字9
原始训练集	11800	5851	5949
测试集	1983	974	1009

4.3 数据预处理

原始图像以 28×28 的 `torch.uint8` 张量保存，像素范围为 $[0, 255]$。预处理后，每张图像均表示为 784 维 `float32` 特征向量，像素范围缩放为 $[0, 1]$。

原始训练集按照 8:2 的比例进行分层随机划分，固定 `random_state=23`，从而使不同模型使用完全相同的训练集与验证集。测试集不参与划分和参数选择，只在最终模型确定后用于独立评价。数据变化及划分结果如下：

数据集	样本数量	预处理后特征形状	特征类型与范围	用途
训练集	9440	$(9440, 784)$	`float32`，$[0, 1]$	模型拟合
验证集	2360	$(2360, 784)$	`float32`，$[0, 1]$	超参数选择
测试集	1983	$(1983, 784)$	`float32`，$[0, 1]$	最终独立评价

五、算法实现与关键代码

5.1 数据预处理

数据预处理由 `data_loader.py` 中的 `load_data()` 函数统一完成，具体流程如下。

1. **加载数据。** 使用 `torch.load()` 分别读取训练集和测试集，设置 `map_location="cpu"`，保证数据在 CPU 环境下可以直接加载；同时设置 `weights_only=True`，只读取张量数据。
2. **数据类型转换。** 将图像张量和标签张量转换为 NumPy 数组，以便后续使用 NumPy 和 Scikit-learn 进行计算。
3. **图像展平。** 原始图像尺寸为 28×28，使用 `reshape(-1, 784)` 将每张图像转换为一个 784 维特征向量。展平后，训练集和测试集均可表示为“样本数 × 784”的二维矩阵。
4. **像素归一化。** 先将图像数组转换为 `np.float32`，再将每个像素值除以 255，使数据范围由 $[0, 255]$ 缩放至 $[0, 1]$：

$$X' = \frac{X}{255}$$

5. **划分训练集和验证集。** 使用 `train_test_split()` 将原始训练集按 8:2 划分为训练集和验证集，其中 `test_size=0.2`、`random_state=23`、`shuffle=True`。设置 `stratify=y_train` 进行分层抽样，使训练集与验证集中数字 8、9 的比例尽量保持一致。划分后训练集包含 9440 个样本，验证集包含 2360 个样本，独立测试集包含 1983 个样本。
6. **按模型进行标签编码。** Scikit-learn 的两种 SVM 直接使用原始标签 8 和 9；Hinge Loss 模型将数字 9 编码为 $+1$、数字 8 编码为 -1；Cross-Entropy 模型将数字 9 编码为 1、数字 8 编码为 0。不同编码分别满足间隔损失和概率损失的计算要求。

数据展平、归一化和标签编码的核心代码如下：

```
X_train = X_train.numpy().reshape(-1, 784)
X_test = X_test.numpy().reshape(-1, 784)

X_train = X_train.astype(np.float32) / 255.0
X_test = X_test.astype(np.float32) / 255.0

y_train_hinge = np.where(y_train == 9, 1, -1)
y_train_ce = np.where(y_train == 9, 1, 0)
```

在超参数选择阶段，仅使用训练集拟合模型、使用验证集比较参数。测试集不参与参数选择，保留到最终模型确定后再进行一次独立评价，从而减少测试信息泄漏。

5.2 模型初始化

本实验对手动实现的模型与调用 Scikit-learn 的 SVM 采用不同的初始化方式。

对于 Hinge Loss 和 Cross-Entropy 两个手写线性模型，权重和偏置在每次调用 `fit()` 时重新初始化。设输入特征数为 $d = 784$，初始化方式为：

$$w^{(0)} = \mathbf{0} \in \mathbb{R}^{784}, \qquad b^{(0)} = 0$$

权重数组的数据类型设为 `np.float32`。随后，模型通过训练数据计算出的梯度逐轮更新参数。由于本实验只有一个线性输出单元，不存在多层神经网络中多个神经元使用相同初值所产生的对称性问题，因此零初始化可以正常使用。

两个手写模型使用相同的参数初始化方式：

```
def initialize(self, n_features):
    self.w = np.zeros(n_features, dtype=np.float32)
    self.b = 0.0
```

Linear SVM 和 RBF SVM 不手动设置 w、b 或支持向量，而是分别创建：

```
SVC(kernel="linear", C=c_value)
SVC(kernel="rbf", C=c_value, gamma=gamma)
```

之后调用 `fit(X_train, y_train)`，由 Scikit-learn 的 `SVC` 求解软间隔 SVM 的优化问题。程序只负责指定核函数及超参数，并通过验证集选择合适的参数组合。

5.3 Linear SVM 训练

Linear SVM 与 RBF SVM 的训练、参数搜索和评价函数均位于 `svm_models.py`。当前实验用于保存完整验证结果并绘制参数搜索图的 Linear SVM 搜索函数为 `search_linear_svm_grid()`，候选惩罚系数设置为：

$$C \in \{0.01, 0.1, 1, 10, 100\}$$

具体训练与选择步骤如下：

1. 依次取一个候选 C，使用 `SVC(kernel="linear", C=C)` 在 9440 个训练样本上拟合模型。
2. 使用训练好的模型预测验证集标签。
3. 将数字 9 设为正类，即 `pos_label=9`，计算验证集 F1-score。
4. 将当前 F1-score 与已记录的最佳值比较；若当前结果更高，则更新 `best_C` 和 `best_f1`。
5. 遍历全部 5 个候选值后，返回验证集 F1-score 最高的 C。

对应的参数搜索核心代码为：

```
for C in C_values:
    model = train_linear_svm(X_train, y_train, C)
    y_pred = model.predict(X_val)
    current_f1 = f1_score(y_val, y_pred, pos_label=9)

    if current_f1 > best_f1:
        best_C = C
        best_f1 = current_f1
```

选择 F1-score 而不是只使用 Accuracy，是因为 F1-score 同时考虑数字 9 的 Precision 和 Recall，能够综合反映误判与漏判情况。实验得到的最优参数为 $C = 0.1$。该参数只由训练集和验证集确定，随后用于最终模型的重新训练。

5.4 RBF SVM 训练

RBF SVM 需要同时确定惩罚系数 C 与核参数 γ。代码设置的搜索空间为：

$$C \in \{0.01, 0.1, 1, 10, 100\}$$

$$\gamma \in \{\text{scale}, 0.001, 0.01, 0.1, 1\}$$

程序使用两层循环进行网格搜索，共训练 $5 \times 5 = 25$ 个候选模型。对每一组 (C, γ)，执行以下步骤：

1. 使用 `SVC(kernel="rbf", C=C, gamma=gamma)` 在训练集上拟合模型。
2. 在验证集上生成预测标签。
3. 以数字 9 为正类计算验证集 F1-score。
4. 若当前 F1-score 高于此前最佳结果，则同时更新最佳 C、最佳 γ 和最佳 F1-score。

网格搜索通过两层循环遍历全部参数组合：

```
for C in C_values:
    for gamma in gamma_values:
        model = train_rbf_svm(X_train, y_train, C, gamma)
        y_pred = model.predict(X_val)
        current_f1 = f1_score(y_val, y_pred, pos_label=9)

        if current_f1 > best_f1:
            best_C = C
```

```
                best_gamma = gamma
                best_f1 = current_f1
```

候选值中的 `gamma="scale"` 表示由 Scikit-learn 根据输入特征数量和训练数据方差自动确定核参数。网格搜索最终选择 $C = 10$、$\gamma = \text{scale}$。与 Linear SVM 相比，RBF SVM 多搜索了一个控制核作用范围的参数，因此能够调整非线性决策边界的复杂程度。

5.5 Hinge Loss 训练

Hinge Loss 的目标函数、次梯度推导及其与软间隔 SVM 的关系已在第 3.4 节说明。本节只说明代码实现：`hinge_model.py` 将数字 9 编码为 $+1$、数字 8 编码为 -1；每轮先计算带符号间隔，再通过 `mask = margin < 1` 选出误分类或虽分类正确但未达到单位间隔的样本。梯度采用布尔掩码和矩阵乘法一次性计算，而不逐样本循环：

```
margin = y * self.decision_function(X)
mask = margin < 1

X_active = X[mask]
y_active = y[mask]

grad_w = -(X_active.T @ y_active) / len(X) \
         + self.reg_lambda * self.w
grad_b = -np.sum(y_active) / len(X)
```

经二维联合网格搜索，Hinge Loss 在学习率 $\eta = 0.4$、正则化系数 $\lambda = 3 \times 10^{-5}$ 时取得最高验证集 F1-score（0.983165）；3×10^{-5} 至 3×10^{-4} 的局部区域表现相同，表明该参数组合附近较稳定。最终训练固定为 1000 轮。每轮参数更新后，程序计算一次完整训练损失并保存至 `loss_history`；若传入验证集，同时将验证损失保存至 `val_loss_history`，用于绘制训练损失与验证损失曲线。训练结束后，模型以决策得分 0 为阈值：得分大于或等于 0 时预测为 $+1$，否则预测为 -1。

5.6 Cross-Entropy 训练

Cross-Entropy 的 Sigmoid、数值稳定处理和梯度推导已在第 3.5 节给出。实现中，`ce_model.py` 将数字 9 编码为 1、数字 8 编码为 0，并在计算损失时根据概率数组的数据类型取得机器精度：

```
eps = np.finfo(p.dtype).eps
p = np.clip(p, eps, 1 - eps)
```

将概率限制在 $[\varepsilon, 1-\varepsilon]$ 内。其中，ε 为当前 `float32` 概率数组的机器精度；这样避免了 `1 - 1e-12` 在 `float32` 中被舍入为 1、进而再次出现 $\log(0)$ 的问题。该截断仅用于损失计算，不会改变预测时采用的 0.5 阈值。

随后以 `error = p - y` 计算误差，并使用矩阵运算实现第 3.5 节推导的梯度：

```
p = self.compute_sigmoid(self.decision_function(X))
error = p - y

grad_w = X.T @ error / len(X) \
         + self.reg_lambda * self.w
grad_b = np.mean(error)
```

然后采用与 Hinge Loss 模型相同的全批量梯度下降形式更新 w 和 b。初步网格搜索后，学习率最优点仍位于候选范围右边界，因此进一步固定 $\lambda = 10^{-5}$，比较 $\eta \in \{1.5, 1.8, 2.0\}$；三者的验证集 F1-score 均为 0.981132。为以更小的更新步长达到相同性能，最终取 $\eta = 1.5$、$\lambda = 10^{-5}$，训练 1000 轮。每轮更新后记录训练损失；提供验证集时同步记录验证损失。预测阶段计算 $p = \sigma(Xw + b)$，当 $p \geq 0.5$ 时预测为数字 9，否则预测为数字 8。

5.7 训练技巧

为提高计算效率、控制内存占用并便于分析训练过程，代码中采用了以下实现技巧。

1. **向量化与 `float32`。** 手写模型使用 `X @ w`、`X.T @ error` 和布尔掩码一次处理全部样本，避免 Python 层逐样本循环；图像、参数和中间计算使用 `float32`，以降低内存占用并满足本实验精度需求。
2. **可复现性控制。** 训练集/验证集固定使用 `random_state=23` 划分；相关脚本将 OpenBLAS、OMP 和 MKL 线程数设为 1；`src/experiment_utils.py` 统一维护最终参数、训练轮数和标签还原规则，避免不同实验入口出现配置不一致。
3. **损失记录与分离入口。** 两个手写模型分别维护训练和验证损失历史；`scripts/final_evaluation.py`、`scripts/run_grid_search.py` 与 `scripts/repeated_validation.py` 分别承担最终评价、单模型参数搜索和重复验证，避免为补充某项分析而重复运行无关模型。

5.8 最终模型训练

完成超参数选择后，验证集已经完成了模型选择任务。为了使最终模型利用更多已标注样本，代码使用 `np.concatenate()` 将训练集与验证集重新合并：

$$X_{\text{full}} = X_{\text{train}} \cup X_{\text{val}}, \qquad y_{\text{full}} = y_{\text{train}} \cup y_{\text{val}}$$

合并后共有 11800 个训练样本。四种模型均使用已经确定的超参数从头重新训练，而不是直接沿用参数搜索阶段只在 9440 个样本上得到的模型。最终训练配置如下：

```
X_full = np.concatenate([X_train, X_val], axis=0)
y_full = np.concatenate([y_train, y_val], axis=0)

final_linear = train_linear_svm(X_full, y_full, 0.1)
final_rbf = train_rbf_svm(X_full, y_full, 10, "scale")
```

模型	最终训练数据	主要超参数	训练方式
Linear SVM	11800	$C = 0.1$	Scikit-learn `SVC` 求解

模型	最终训练数据	主要超参数	训练方式
RBF SVM	11800	$C = 10$，$\gamma = \text{scale}$	Scikit-learn SVC 求解
Hinge Loss	11800	$\eta = 0.4$，$\lambda = 3 \times 10^{-5}$，1000 轮	全批量梯度下降
Cross-Entropy	11800	$\eta = 1.5$，$\lambda = 1 \times 10^{-5}$，1000 轮	全批量梯度下降

SVM 使用原始的 8、9 标签重新训练；Hinge Loss 和 Cross-Entropy 则分别将合并后的标签转换为 $\{-1, +1\}$ 和 $\{0, 1\}$。训练完成后，四种模型在此前未参与训练和调参的 1983 个测试样本上生成预测结果，并计算 Accuracy、Precision、Recall、F1-score 及 $8 \to 9$、$9 \to 8$ 两类错分数量。至此，整个流程实现了"训练集拟合—验证集选参—训练集与验证集合并重训—独立测试集评价"的完整实验设计。

六、实验结果、分析与讨论

本章对四种分类模型的实验结果进行分析。首先比较 Linear SVM 与 RBF SVM，以考察核函数对分类能力的影响；随后比较手动实现的 Hinge Loss 分类器与 Linear SVM，并讨论 Hinge Loss 和 Cross-Entropy 两种损失函数产生的差异；最后结合超参数敏感性、损失曲线、错分方向统计和多项评价指标进行综合分析。

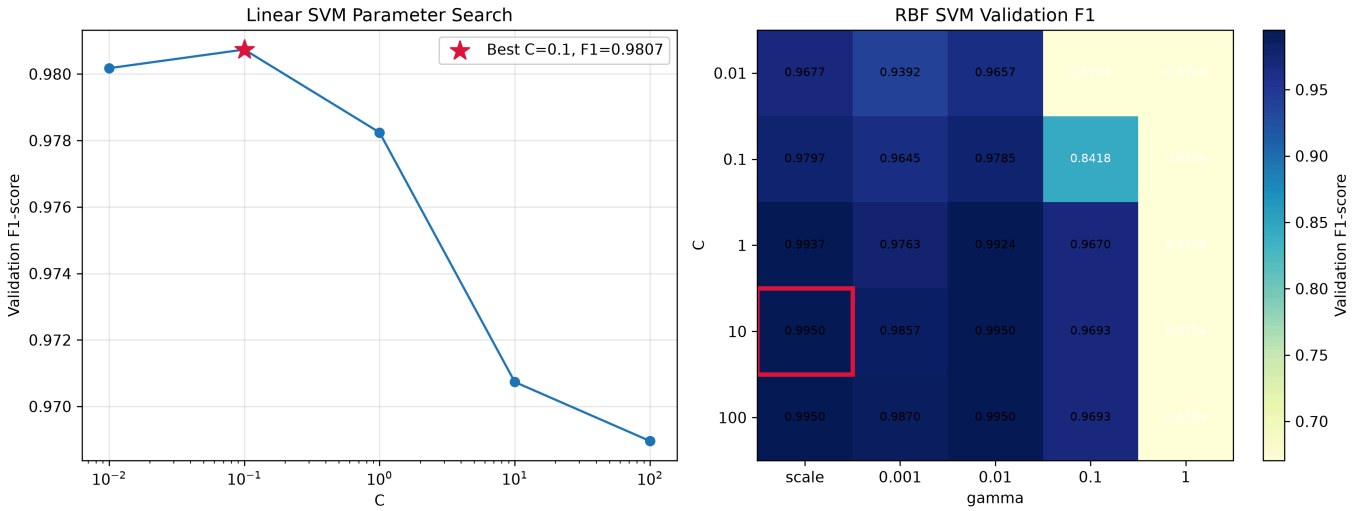
本实验将数字 9 作为正类、数字 8 作为负类。超参数均根据验证集 F1-score 选择，测试集不参与调参。参数确定后，使用原始训练集与验证集合并得到的全部 11800 个样本重新训练最终模型，再在包含 1983 个样本的独立测试集上评价。因此，以下测试结果主要反映模型在当前数据集和实验设置下的泛化表现。

6.1 不同核函数的 SVM 性能比较

6.1.1 参数选择与测试结果

Linear SVM 只需搜索惩罚参数 C；RBF SVM 除 C 外，还需搜索控制核函数作用范围的 γ。两种模型均以验证集 F1-score 作为选择标准，最终结果如下：

模型	最终参数	验证集 F1-score
Linear SVM	$C = 0.1$	0.980737
RBF SVM	$C = 10$，$\gamma = \text{scale}$	0.994954



Linear SVM 的最佳 $C = 0.1$ 位于候选范围内部。RBF SVM 中，$(C = 10, \gamma = \text{scale})$、$(10, 0.01)$、$(100, \text{scale})$ 与 $(100, 0.01)$ 的验证集 F1-score 均为 0.994954。代码采用严格的 > 比较更新最佳值，因此保留按遍历顺序首先达到最大值的 $(C = 10, \gamma = \text{scale})$ 作为最终配置。图中同时展示了其余候选参数的表现，为最终参数选择提供了依据。

使用上述参数在全部 11800 个训练样本上重新训练后，RBF SVM 的测试集 F1-score 为 99.60%，比 Linear SVM 的 98.06% 高约 1.54 个百分点；完整指标见第 6.6.3 节。该差距对应的错误样本数变化较为明显：Linear SVM 错分 39 个样本，而 RBF SVM 仅错分 8 个样本。

6.1.2 核函数差异分析

Linear SVM 直接在 784 维像素特征空间中寻找线性决策超平面。如果数字 8 和数字 9 的样本分布可以被一个线性边界较好地分开，线性模型就能获得较高性能；但其表达能力受限于线性形式，难以描述复杂的局部边界。

手写数字存在明显的类内差异。同一个数字可能因书写者不同而表现出笔画粗细、封闭区域大小、倾斜方向和局部连接方式的变化；与此同时，部分数字 8 与数字 9 的局部形态可能十分接近。这意味着两类样本在原始像素空间中未必完全线性可分。RBF 核根据样本间距离度量相似性，可在隐式高维空间中构造非线性决策边界，因此能够更细致地适应这类局部结构。

从测试结果看，RBF SVM 将数字 8 误判为 9 的数量由 17 降至 5，将数字 9 误判为 8 的数量由 22 降至 3，说明其提升并非只来自对某一类别的偏向，而是同时改善了两类样本的识别。尽管如此，本实验只能说明 RBF 核在当前数据、特征表示和参数范围下更合适，不能据此推断 RBF 核在所有任务中都优于线性核。非线性核通常还需要更多计算，并对 C 与 γ 的选择更加敏感。

6.2 Hinge Loss 线性分类器与 Linear SVM 的比较

6.2.1 理论联系与实现差异

Hinge Loss 与软间隔 Linear SVM 的理论关系见第 3.4 节。二者均体现最大间隔思想，但本实验的手写模型采用固定学习率的全批量次梯度下降，Scikit-learn SVC 使用专用优化求解器；二者的损失缩放、超参数含义和停止条件不同，因此不要求得到完全相同的参数或预测结果。

6.2.2 实验结果比较

两者的完整指标见第 6.6.3 节。采用联合网格搜索后的 Hinge Loss 分类器 F1-score 为 98.46%，比 Linear SVM 高约 0.40 个百分点；错分数量由 39 降至 31。结合错分方向统计，Hinge Loss 将数字 8 错判为 9 的样本数由 17 降至 13，将数字 9 错判为 8 的样本数由 22 降至 18，说明当前参数同时降低了两类错误。

这一结果说明，手动实现的 Hinge Loss 分类器已经复现了线性最大间隔模型的主要行为，并达到与库模型相近的分类能力；两者的差别主要体现为决策边界位置和两类错误的分配不同，而不能简单理解为某个模型在所有方面都更优。

6.3 Hinge Loss 与 Cross-Entropy 线性分类器比较

6.3.1 参数与测试结果

两种手动实现模型的最终训练设置如下：

模型	学习率	正则化系数	训练轮数
Hinge Loss	0.4	3×10^{-5}	1000
Cross-Entropy	1.5	1×10^{-5}	1000

完整测试指标见第 6.6.3 节。Hinge Loss 的 F1-score 为 98.46%，略高于 Cross-Entropy 的 98.31%，总错分数分别为 31 与 34。该差异幅度有限，仍应结合损失函数和优化过程理解。

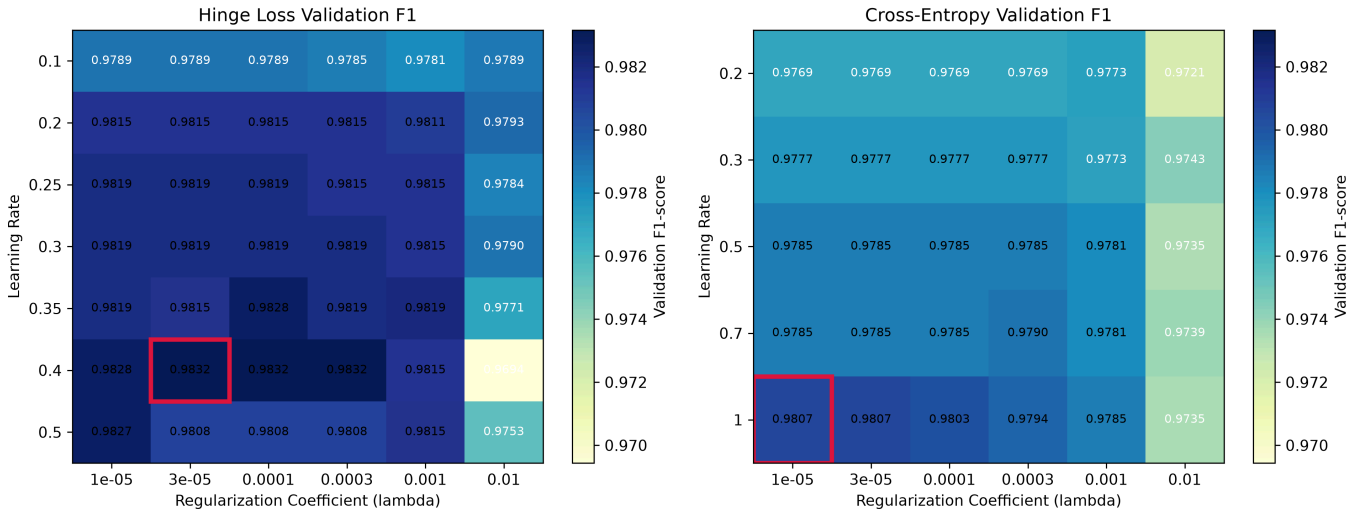
6.3.2 损失函数差异分析

两种模型都使用线性决策函数 $w^T x + b$，因此它们具有相同层次的模型表示能力，主要区别来自优化目标。Hinge Loss 关注带符号间隔：样本一旦被正确分类且间隔达到 1，其经验损失和相应梯度就变为 0，训练重点集中在误分类样本和靠近边界的样本上。Cross-Entropy 则通过 Sigmoid 输出概率，只要预测概率与真实标签不完全一致，样本通常仍会产生梯度；对于高置信度的错误预测，其惩罚尤其明显。

本次实验中，Hinge Loss 得到的边界在两个错误方向上都略占优势；但两种模型的 F1-score 仅相差约 0.15 个百分点。这一差异既反映损失函数对边界样本的处理方式，也受到学习率、正则化强度和有限训练轮数影响。由于二者使用的最佳超参数并不相同，不能仅凭本次结果得出 Hinge Loss 在一般分类任务中必然优于 Cross-Entropy 的结论。

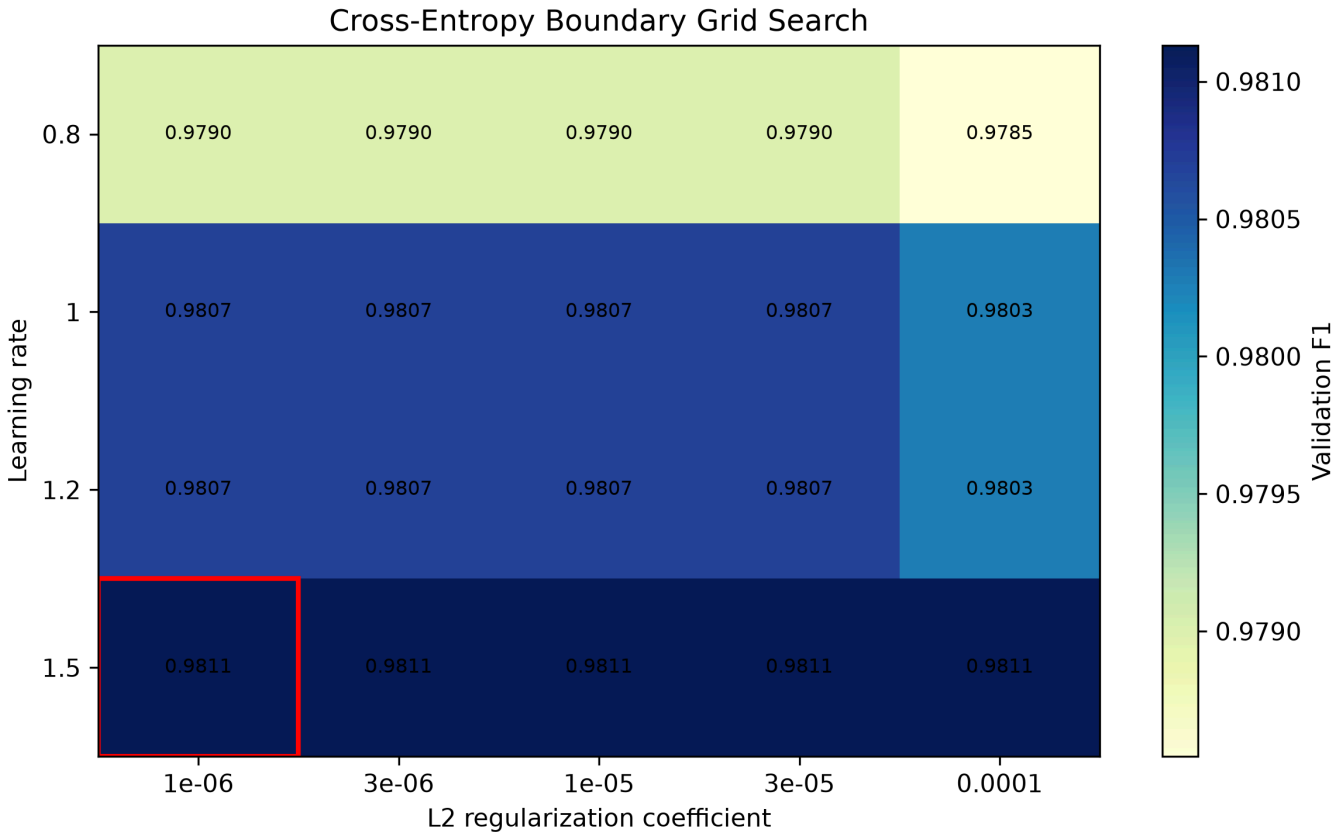
6.4 联合网格搜索与参数敏感性分析

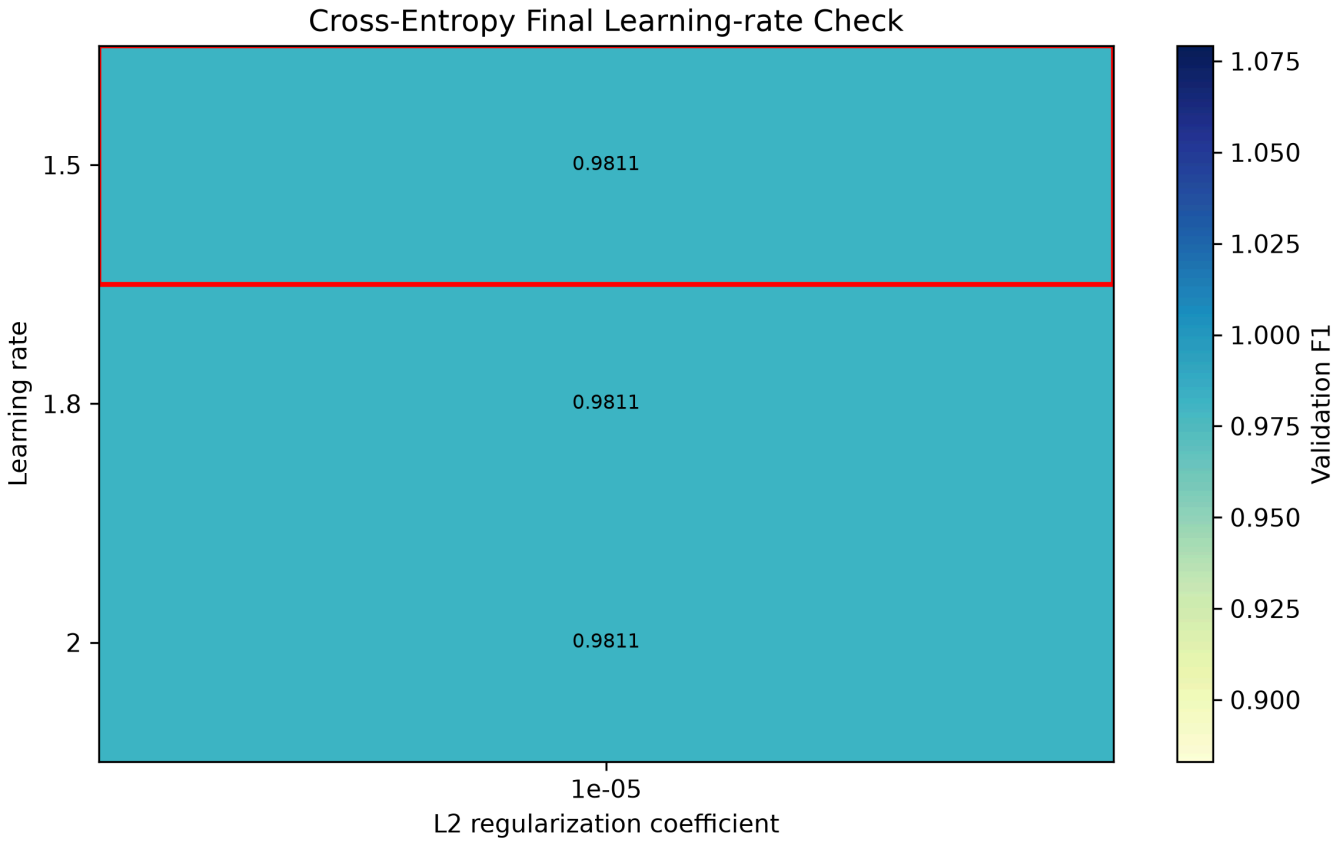
手写模型的学习率与正则化系数会共同影响更新轨迹，因此本实验采用二维网格，以验证集 F1-score 选择参数，而非分别固定另一参数做一维比较。搜索过程及结果见图 6-2。



Hinge Loss 在 $\eta = 0.4$、$\lambda = 3 \times 10^{-5}$ 时取得最高验证集 F1-score 0.983165；当 λ 取 3×10^{-5}、10^{-4} 与 3×10^{-4} 时结果相同，说明最优区域并非孤立点。相较于较小学习率，$\eta = 0.4$ 在固定 1000 轮内能够更充分优化；继续增大正则化则会逐渐削弱分类能力。

Cross-Entropy 的初步网格搜索中，最高 F1-score 为 0.980688，且学习率最优点位于 1.0 的右边界。为避免将边界点误判为最优点，进一步在 $\eta \in \{0.8, 1.0, 1.2, 1.5\}$、$\lambda \in \{10^{-6}, 3 \times 10^{-6}, 10^{-5}, 3 \times 10^{-5}, 10^{-4}\}$ 上进行局部扩展搜索。结果表明，$\eta = 1.5$ 时五个正则化系数均达到 F1-score 0.981132。随后固定 $\lambda = 10^{-5}$，继续检验 $\eta = 1.5, 1.8, 2.0$，三者结果仍相同。因此选择达到平台区的最小学习率 $\eta = 1.5$，以较保守的更新幅度获得相同性能；正则化系数取位于该平台内部的 10^{-5}。

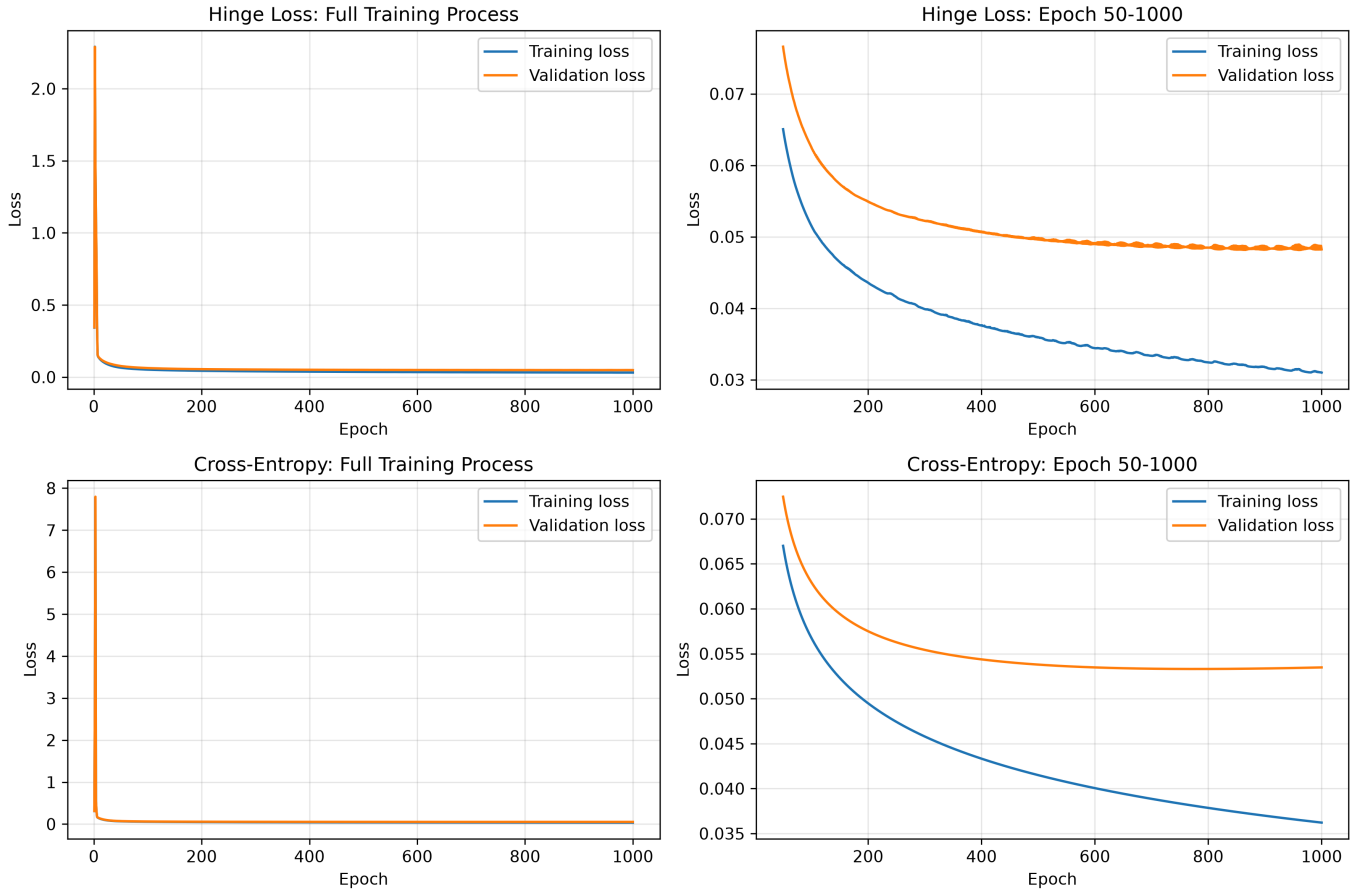




这一过程表明，参数选择不仅应报告当前候选范围内的最大值，还应检查最优点是否落在搜索边界。对 Cross-Entropy 的补充搜索避免了过早固定学习率 1.0，也使最终参数选择更具依据。

6.5 损失函数收敛性分析

训练过程中记录了两个手写模型在 1000 轮内的训练损失与验证损失，结果如图 6-5 所示。图的左列展示完整过程，右列单独放大第 50 至第 1000 轮，避免首轮损失较大而掩盖后期收敛细节。



两种模型的损失均在前期下降较快，之后下降速度逐渐减慢，说明梯度下降能够有效降低各自的目标函数。放大图显示，Cross-Entropy 的训练损失持续下降，验证损失在后期稳定于约 0.053 附近，未出现发散；Hinge Loss 的训练损失持续下降，验证损失由约 0.075 逐步降至约 0.048，末段存在轻微的非单调变化。由于本实验采用确定性的全批量梯度下降，并不存在小批量采样噪声，这些起伏不宜称为“随机波动”；它们更可能与 Hinge Loss 的分段形式、活跃样本集合随迭代改变及有限数值精度有关。两者的训练损失均低于验证损失，但间隔没有持续扩大。

训练损失始终低于验证损失是正常现象，因为模型参数直接依据训练样本优化。两者之间存在一定间隔，但没有出现训练损失持续显著下降而验证损失大幅上升的情况，因此当前曲线没有显示出严重过拟合。不过，Hinge Loss 后期验证损失改善已经非常有限，说明继续增加训练轮数未必能带来相应的泛化收益，可在后续实验中考虑根据验证损失设置早停条件。

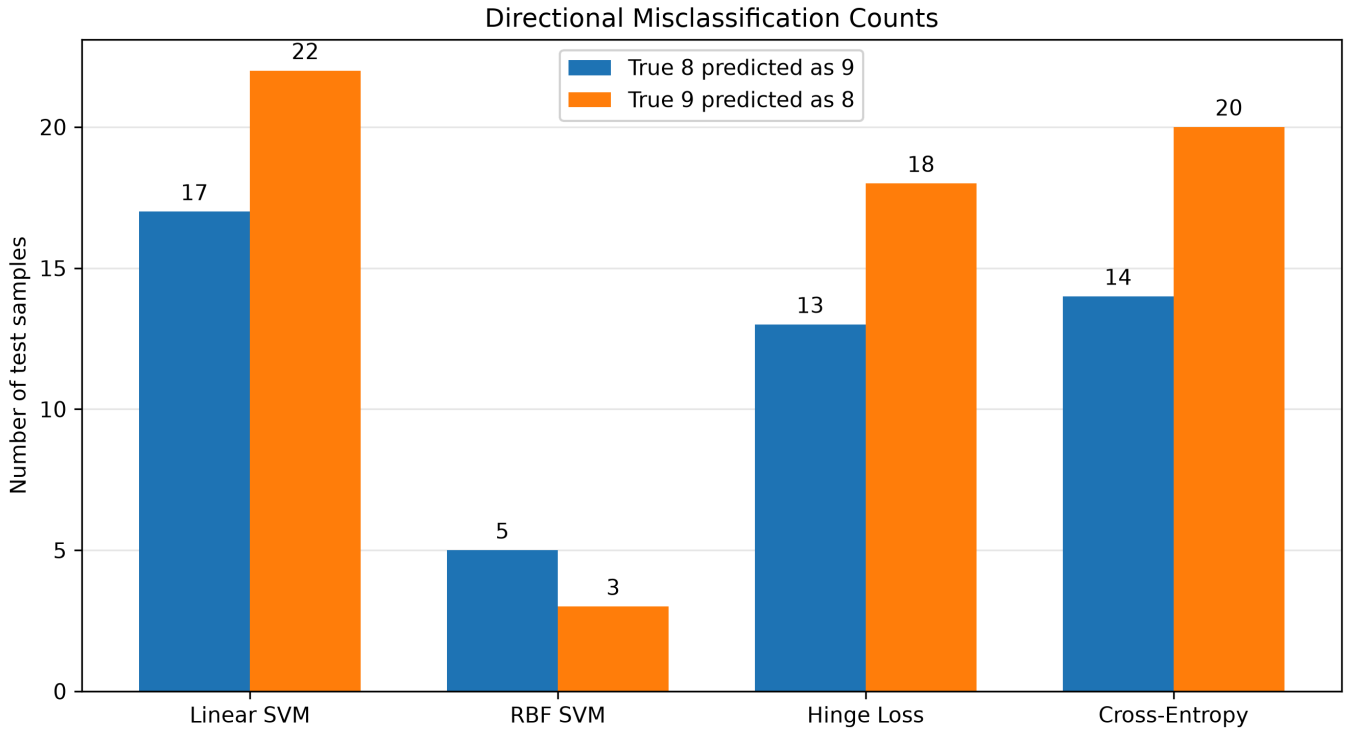
需要强调的是，Hinge Loss 与 Cross-Entropy 的定义和数值尺度不同，图中最终损失值不能直接用于判断哪个模型分类性能更好。损失曲线主要用于分析各自的优化过程，模型优劣仍应结合验证集和测试集上的分类指标判断。

6.6 四种模型的综合性能比较

6.6.1 错分方向统计

根据最终测试集预测，可将两种错误类型汇总如下：

模型	数字 8 误判为 9（FP）	数字 9 误判为 8（FN）	总错误数
Linear SVM	17	22	39
RBF SVM	5	3	8
Hinge Loss	13	18	31
Cross-Entropy	14	20	34

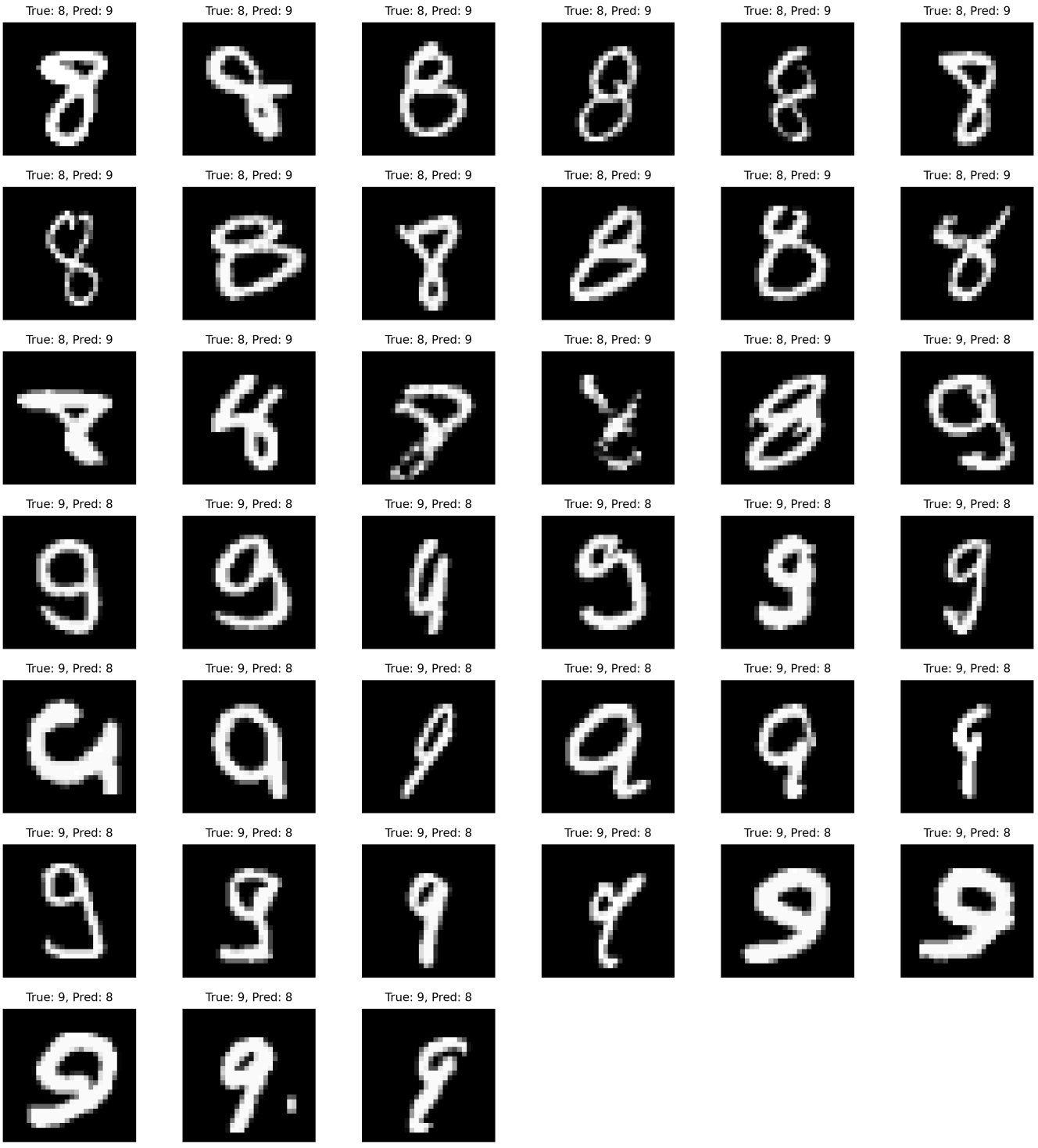


RBF SVM 在两种错误类型上的数量都最少，说明其高 Accuracy 和 F1-score 并非由偏向某一类别造成。三个线性模型中，Hinge Loss 的总错分数最少。三种线性模型均更容易将数字 9 误判为数字 8，说明某些 9 的闭环或竖直笔画特征较弱时，展平像素构成的线性边界更易将其归入数字 8。

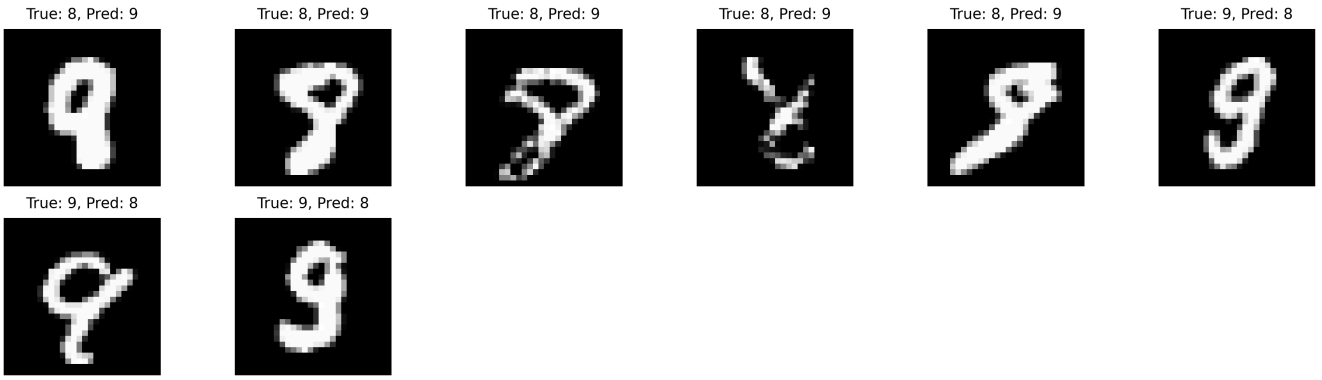
6.6.2 全部错分样本与分类错误原因分析

程序按错误方向依次导出了每个模型在测试集上的全部错分样本，样本标题标明真实标签和预测标签；每张图先排列 $8 \to 9$ 的错误，再排列 $9 \to 8$ 的错误。这样可将前述错分数量统计与具体图像形态对应起来。

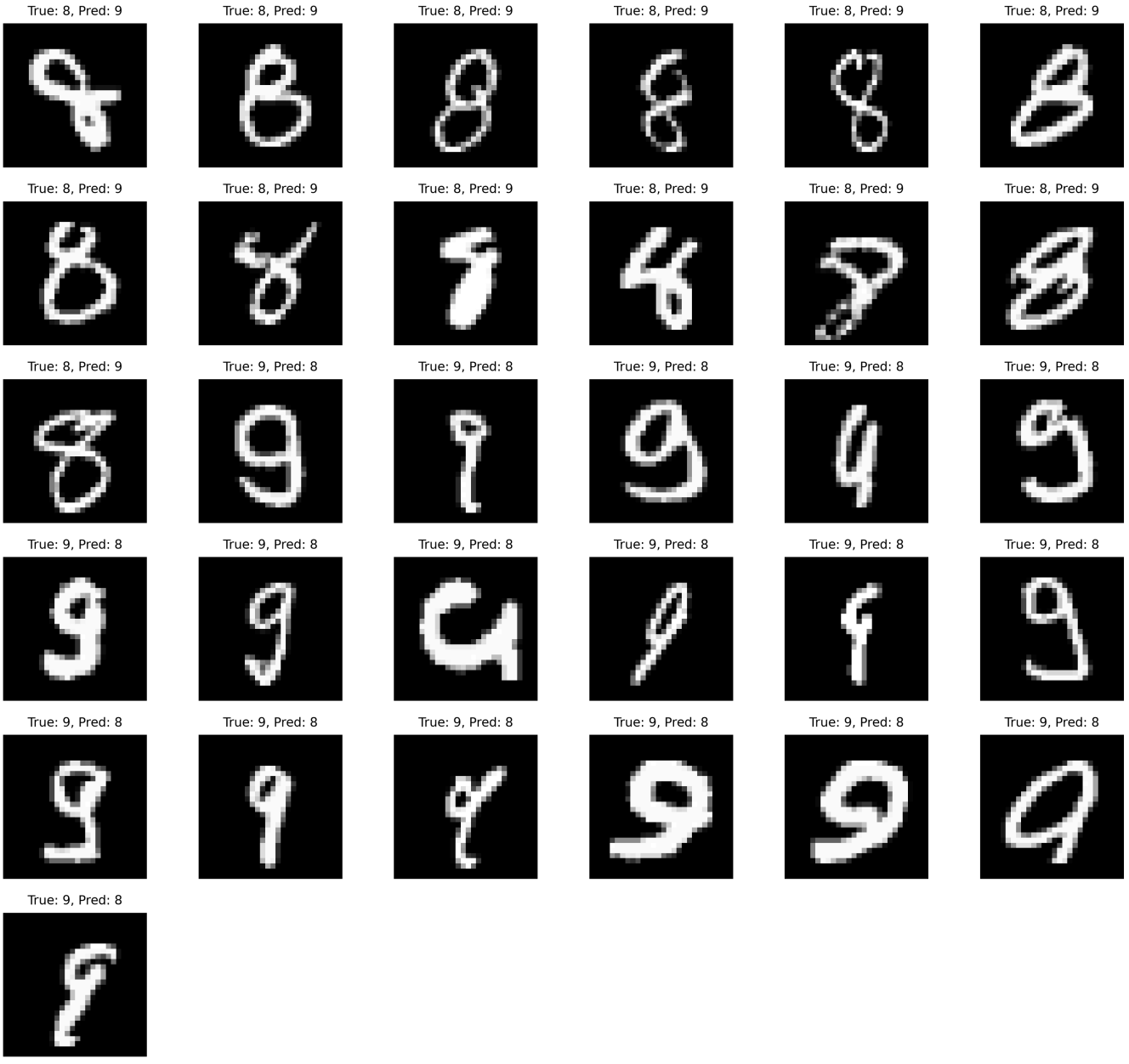
All Misclassified Test Samples: Linear SVM (8→9: 17, 9→8: 22)

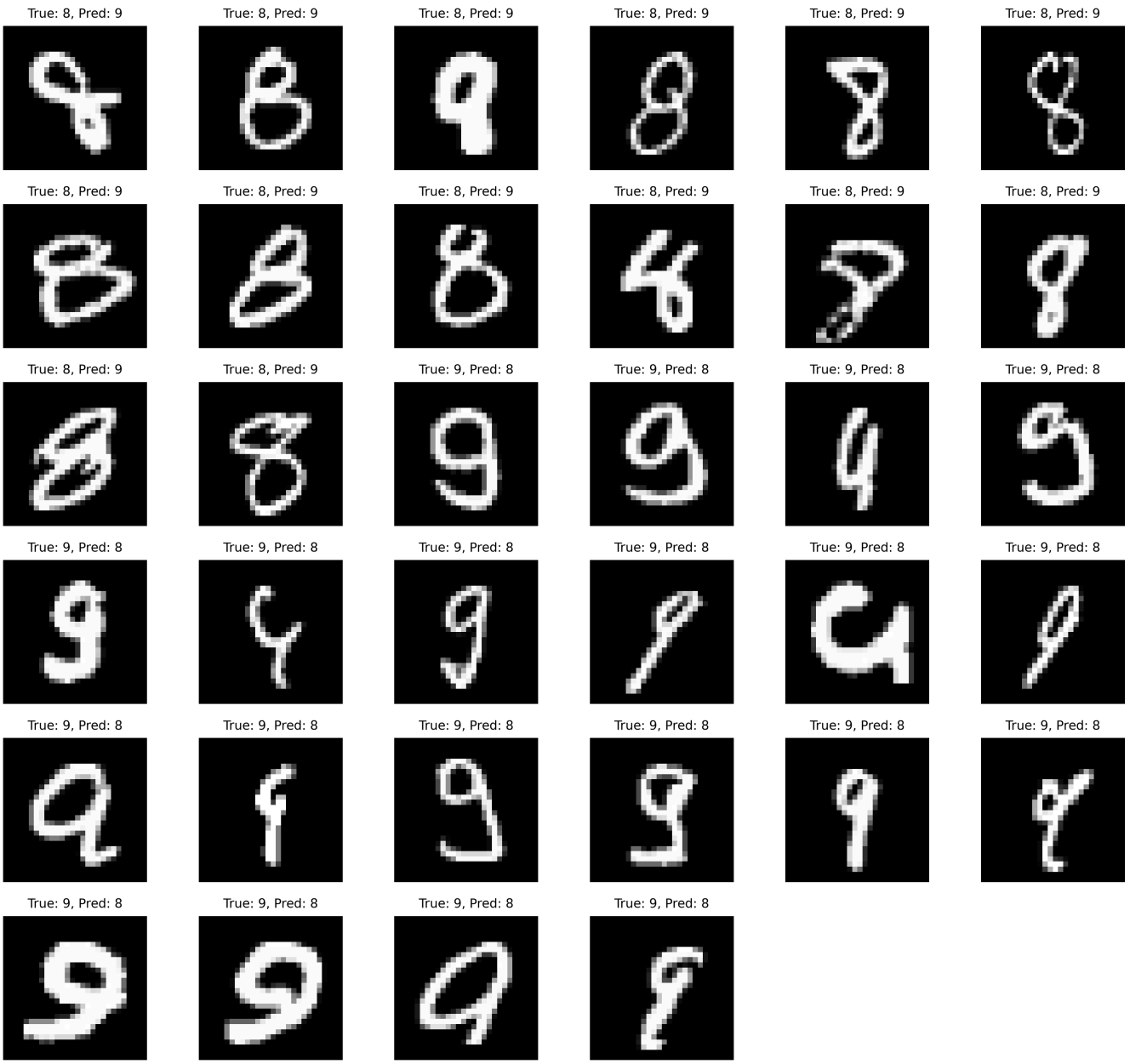


All Misclassified Test Samples: RBF SVM (8→9: 5, 9→8: 3)



All Misclassified Test Samples: Hinge Loss (8→9: 13, 9→8: 18)





从错误方向的比例看，三种线性模型都略偏向发生 $9 \to 8$ 的错误：Linear SVM 的该类错误占 $22/39 = 56.4\%$，Hinge Loss 为 $18/31 = 58.1\%$，Cross-Entropy 为 $20/34 = 58.8\%$。这与三者的 Recall 均略低于 Precision 相吻合，即它们相对更容易漏检真实数字 9。RBF SVM 的错误总数仅为 8，其中 $8 \to 9$ 有 5 个、$9 \to 8$ 有 3 个；由于样本数很小，不能据此断言它存在稳定的错误方向偏好。

对于 $9 \to 8$，图中可观察到的典型情形是：数字 9 的右侧竖直笔画较短、较淡、向左弯折，或与上半环连接位置不清晰。此时原本用于区分 9 的“上环加右侧尾部”结构减弱，整体轮廓更接近上下两个环相连的数字 8。还有少量数字 9 存在明显倾斜、笔画断裂或下端回钩不完整等情况，使尾部难以形成稳定的判别特征。对采用展平像素的线性模型而言，这类局部笔画位置和强度的变化会直接改变多个维度的像素值，从而更容易跨越线性决策边界。

对于 $8 \to 9$，一些数字 8 的下半环存在开口、下环较小，或上下两个环大小差异明显；另一些样本的右侧局部笔画较突出，形成类似数字 9 竖直尾部的结构。这些特征会削弱“两个完整闭环”这一数字 8 的典型外观，并使模型倾向于将其归入 9。局部粘连、笔画过粗或过细也会改变两个环之间的分隔区域，导致模型难以仅凭像素强度判断其真实拓扑结构。

从模型间比较看，Hinge Loss 相比 Linear SVM 同时减少了两类错误（$17 \to 13$ 与 $22 \to 18$），说明在当前参数下，以间隔为目标的手写模型对部分边界样本具有更合适的划分。Cross-Entropy 的总错误数为 34，介于 Hinge Loss 与 Linear SVM 之间，且其 $9 \to 8$ 错误仍多于 $8 \to 9$。RBF SVM 仅有 8 个错分，图中的剩余错误大多是形态本身接近另一类别、笔画不完整或书写高度变形的样本；这说明 RBF 核能够刻画多数非线性边界，但不能完全克服原始图像中固有的形态歧义。

需要说明的是，上述解释来自对全部错分图像的定性观察，并未额外标注"开口""倾斜""粘连"等视觉属性，也未记录样本的决策分数。因此，它们应被表述为与错误模式相一致的可能原因，而不是对每个样本的唯一因果结论。若需要进一步验证，可在后续实验中人工标注这些形态属性，或比较模型的决策分数与预测置信度。

6.6.3 综合评价指标比较

四种模型的 Accuracy、Precision、Recall 和 F1-score 汇总如下：

模型	Accuracy	Precision	Recall	F1-score
Linear SVM	98.03%	98.31%	97.82%	98.06%
RBF SVM	**99.60%**	**99.51%**	**99.70%**	**99.60%**
Hinge Loss	98.44%	98.71%	98.22%	98.46%
Cross-Entropy	98.29%	98.60%	98.02%	98.31%

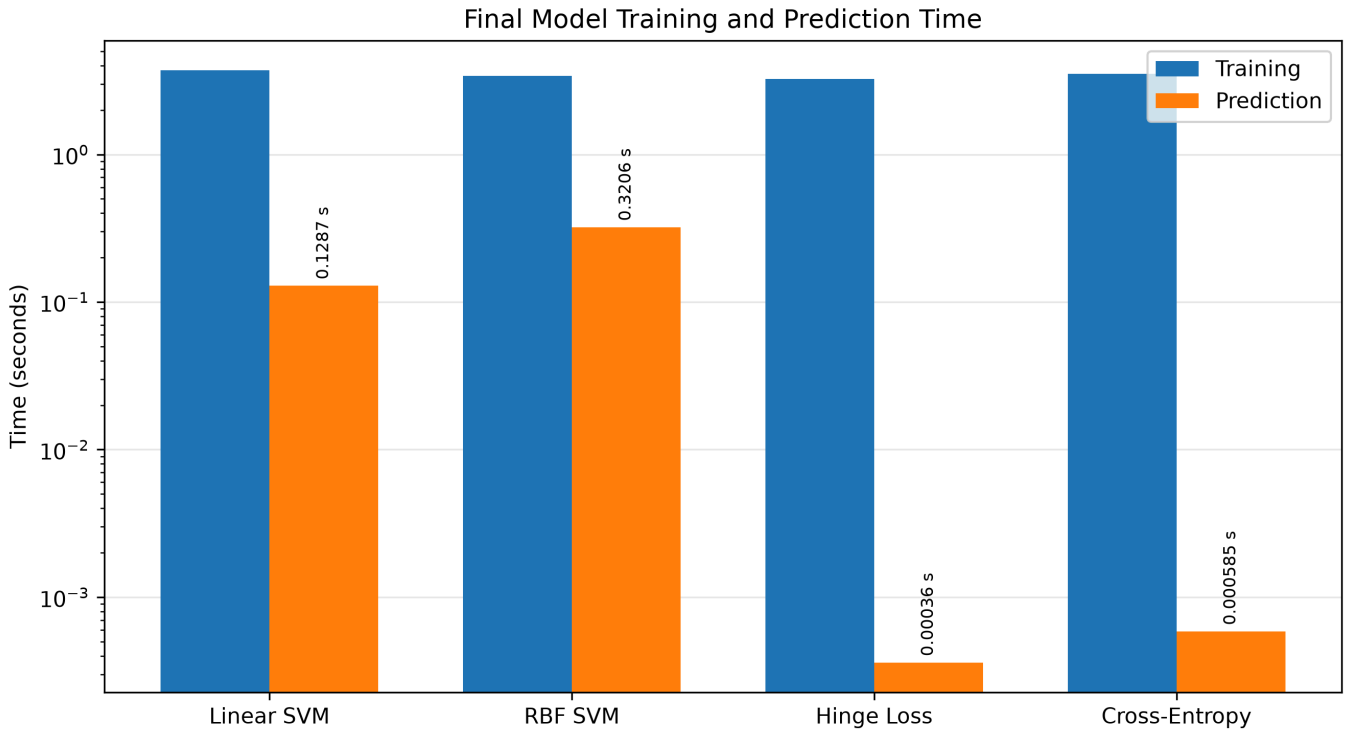
RBF SVM 在四项指标上均为最高，是本次实验中综合性能最好的模型。其 F1-score 比表现最好的线性模型 Hinge Loss 高约 1.14 个百分点，总错误数则由 31 个降至 8 个，说明非线性核带来的改进具有实际意义。

三种线性模型的 Accuracy 均在 98% 左右，F1-score 的最大差距为约 0.40 个百分点（Hinge Loss 为 98.46%，Linear SVM 为 98.06%），表明数字 8 与数字 9 在展平像素特征上已经具有较强的线性可分性，但不同优化目标与求解方式仍会带来可观察的边界差异。Hinge Loss、Cross-Entropy 与 SVC 的优化方式不同，却得到接近的整体性能，也说明线性决策边界是它们共同的主要能力限制。RBF SVM 进一步提升性能，则说明剩余难分样本中存在可被非线性边界利用的结构。

6.6.4 训练与预测效率

在单线程限制下，对合并训练集上的最终训练和 1983 个测试样本的预测分别计时，结果如表所示。计时会受硬件、CPU 负载、Python 与底层数值库版本影响，因此主要用于同一运行环境下的相对比较。

模型	训练时间（s）	测试集预测时间（s）
Linear SVM	3.723517	0.128675
RBF SVM	3.416592	0.320558
Hinge Loss	3.239882	0.000360
Cross-Entropy	3.511538	0.000585



四种模型的训练时间均约为 3 至 4 秒，差异不大；RBF SVM 的预测时间则最长。两个手写线性模型只需一次矩阵乘法和阈值判断，在本次测试中预测耗时约为 3×10^{-4} 秒，远低于两种 SVM。因而，若部署场景特别重视批量预测延迟，手写线性模型具有明显效率优势；若更重视分类性能，则 RBF SVM 仍是更合适的选择。

该耗时比较仍有以下限制。第一，每个模型仅计时一次，未报告多次运行的平均值与标准差，结果可能受到 CPU 短时负载、缓存状态和首次调用开销影响。第二，所有计时均在当前本地软硬件环境和单线程限制下获得，不能直接外推到其他 CPU、并行配置或 GPU 环境。第三，测试集仅有 1983 个样本，预测时间很短；当数据规模变化时，SVM 与矩阵运算的时间增长规律也可能不同。因此，本节结论应理解为当前实验环境、当前数据规模下的相对效率证据，而不是普适的运行时间结论。

6.7 实验讨论与小结

RBF SVM 在四项指标上均为最高，F1-score 为 99.60%，并将总错分数降至 8，说明当前展平像素特征中仍存在可由非线性边界利用的结构。三个线性模型的 F1-score 均在 98% 左右；其中 Hinge Loss 略高于 Linear SVM 和 Cross-Entropy，但差异有限，不能脱离具体参数和数据分布作绝对优劣判断。

损失曲线显示两个手写模型均能稳定优化，未出现明显发散或严重过拟合；耗时比较则表明 RBF SVM 的预测代价更高。结合验证集选参与效率分析，本实验说明模型表示能力、优化目标、超参数与运行约束共同决定最终模型选择。

七、补充验证：多随机种子重复实验

第六章已经从参数搜索、收敛过程、错分样本和运行效率等角度解释了单次固定划分下的实验结果。本章不重复这些分析，而是通过改变训练集/验证集的随机划分，检验固定参数模型的性能排序是否具有稳定性。

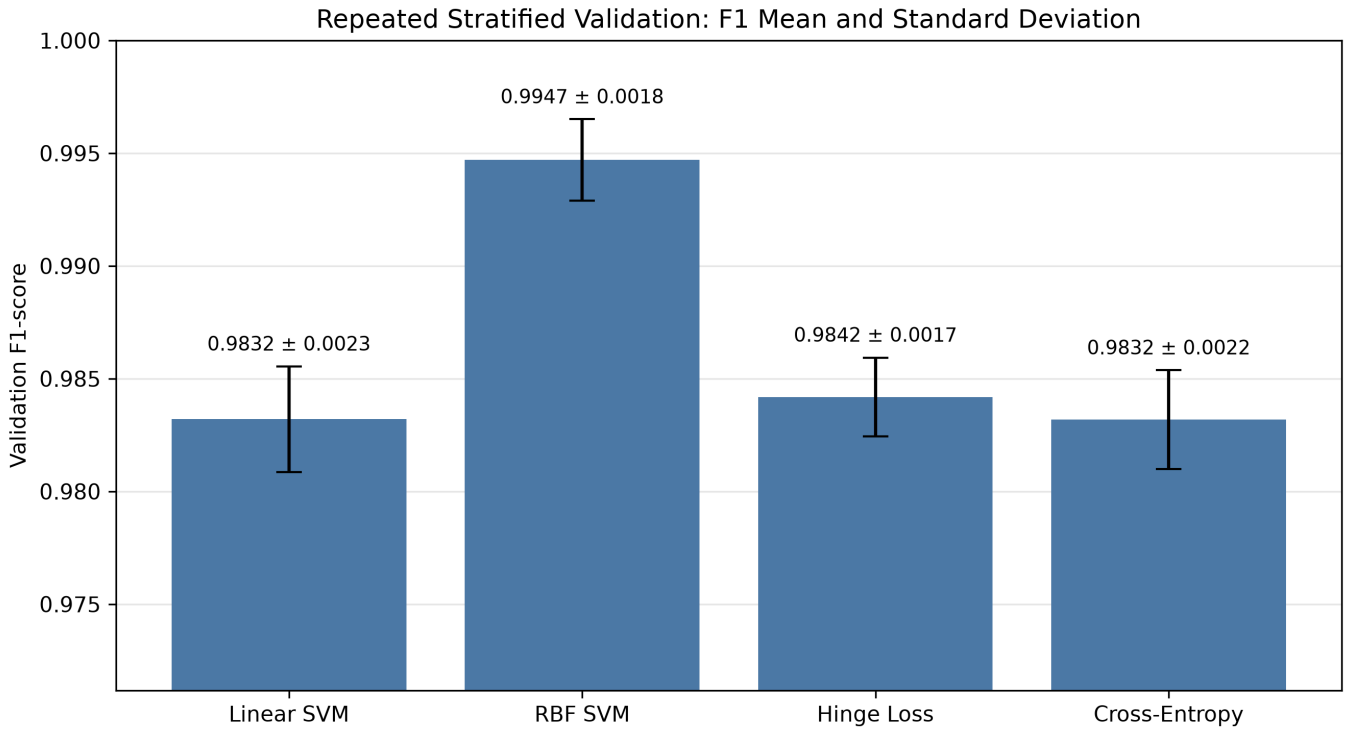
7.1 实验设计

为检验单次训练/验证划分是否带来偶然性，本实验仅使用全部 11800 个原始训练样本，以随机种子 23、42、123、2024 和 2025 分别进行 80%/20% 的分层划分。四种模型均固定使用已确定的最终超参数，不重新进行网

格搜索；独立测试集不加载、不参与本项实验。该设计考察的是固定模型配置对数据划分变化的敏感程度。

7.2 重复验证结果

图 7-1 展示五次验证 F1-score 的均值和样本标准差，表中同时给出 Accuracy、Precision 和 Recall 的均值±标准差。



模型	Accuracy（均值±标准差）	Precision（均值±标准差）	Recall（均值±标准差）	F1-score（均值±标准差）
Linear SVM	0.983051 ± 0.002378	0.982731 ± 0.004774	0.983697 ± 0.003692	0.983204 ± 0.002337
RBF SVM	**0.994661 ± 0.001813**	**0.996458 ± 0.001100**	**0.992941 ± 0.003122**	**0.994695 ± 0.001810**
Hinge Loss	0.984068 ± 0.001762	0.985038 ± 0.003967	0.983361 ± 0.004004	0.984188 ± 0.001746
Cross-Entropy	0.983051 ± 0.002222	0.983705 ± 0.004670	0.982689 ± 0.003545	0.983187 ± 0.002185

7.3 稳定性分析与结论

在五个已测试的划分中，RBF SVM 的 F1-score 每次均为四个模型中最高；其平均值为 0.994695，较平均表现最好的线性模型 Hinge Loss（0.984188）高 0.010507，即约 1.05 个百分点。RBF SVM 的 F1 样本标准差为 0.001810，表明其在这些划分下保持较小波动，但不应据此断言其波动一定小于所有模型：Hinge Loss 的标准差为 0.001746，略低于 RBF SVM。

三种线性模型的平均 F1 均集中在 0.9832 至 0.9842 之间，彼此差异约为千分之一量级，且小于或接近各自的样本标准差；因此，重复验证支持它们整体性能接近，但不足以据此作出三种线性方法之间严格的统计显著性排

序。总体而言，重复实验支持第六章的主要结论：在当前数据与固定参数设置下，RBF SVM 的性能优势并非仅来自单次划分的偶然结果。

7.4 局限性

本实验固定最终超参数，因此考察的是固定模型配置对数据划分变化的稳定性，而不是将完整调参过程嵌入每次划分的嵌套交叉验证。同时，随机种子 23 既用于初始超参数选择，也出现在五次重复划分中，故该次结果并非与调参过程完全独立；重复验证不应被解释为无偏的泛化误差估计。

八、改进方向与不足

8.1 优化方法与训练策略

本实验的参数搜索仍基于有限候选范围，手写模型则采用固定学习率的全批量梯度下降。后续可采用分层交叉验证、随机搜索或贝叶斯优化提高选参可靠性，并比较 SGD、小批量梯度下降和 Adam 等优化方法在训练速度、收敛过程与分类性能上的差异；也可结合验证损失引入早停或学习率调度。

8.2 模型结构与特征改进

当前方法将图像展平为 784 维向量，未利用二维空间结构，且只采用归一化预处理。后续可尝试卷积神经网络等能够提取局部特征的模型，并研究平移、旋转等数据增强或其他图像预处理方法对泛化能力的影响。

8.3 评价与效率分析

本实验已报告五个随机种子下分类指标的均值和标准差，并比较了单次训练与预测耗时；但耗时仍受当前硬件、线程限制和软件版本影响，尚未报告多次重复的时间均值、标准差、内存占用或模型大小。后续可在不同数据规模和重复运行条件下补充这些指标，使模型比较更加全面。

九、实验总结

本实验以手写数字 8 和 9 为对象，完成了 Linear SVM、RBF SVM、手写 Hinge Loss 分类器和手写 Cross-Entropy 分类器的二分类比较。实验覆盖数据预处理、验证集选参、最终模型重训、独立测试、损失记录、错分分析和多随机种子重复验证，形成了从模型实现到结果评价的完整流程。

在当前数据、特征表示与参数范围下，RBF SVM 的测试集 F1-score 为 99.60%，为四种模型中最高；Hinge Loss 和 Cross-Entropy 两个手写线性模型的 F1-score 分别为 98.46% 与 98.31%，预测耗时显著低于两种 SVM。重复验证进一步支持 RBF SVM 的性能优势并非单次划分的偶然现象。总体而言，本实验说明模型的表示能力、损失函数、优化方式、超参数和效率约束共同影响最终模型选择。

附录：实验代码与文件结构

依据作业要求，最终压缩包只包含实验报告和实验代码，不包含教师提供的数据、实验原始结果、虚拟环境或 Git 文件：

```
学号+姓名/
│
├── 实验报告.pdf                 # 图、表和公式均嵌入该文件
├── README.md
├── requirements.txt
├── src/                         # 数据处理与模型实现
│   ├── __init__.py
│   ├── data_loader.py
│   ├── svm_models.py
│   ├── hinge_model.py
│   ├── ce_model.py
│   ├── hyperparameter_search.py
│   └── experiment_utils.py      # 最终参数、统一训练和标签还原
├── scripts/                     # 推荐直接运行的实验脚本
│   ├── check_data.py
│   ├── run_grid_search.py
│   ├── final_evaluation.py
│   ├── refresh_analysis_figures.py
│   └── repeated_validation.py
```

报告中的图、表和实验结论均已在 `实验报告.pdf` 中呈现，因此不需要额外提交 `results/` 目录中的 CSV 与图片；代码运行所需的教师数据也不随压缩包提交。

实验复现说明：

- 数据集由教师提供，提交压缩包时不包含数据；复现实验时，应将 `training.pt` 和 `test.pt` 放置在 `data/processed_8_9/` 目录下。
- 环境版本为 Python 3.13.12、NumPy 1.26.4、PyTorch 2.12.0+cu126、Scikit-learn 1.8.0、Matplotlib 3.10.9。
- 常用实验命令如下：

```
1. 网格搜索：python scripts/run_grid_search.py --model <模型名>
2.Cross-Entropy 最终边界确认：python scripts/run_grid_search.py --model ce --ce-final-check
3.四种最终模型的训练、测试与分析图表生成：python scripts/final_evaluation.py
```

- 多随机种子重复分层验证可运行 `python scripts/repeated_validation.py`；该脚本只读取原始训练集，不加载测试集，输出五次验证的明细、均值±标准差汇总及 F1 稳定性图。最终参数、训练轮数和手写模型标签还原逻辑由 `src/experiment_utils.py` 统一维护，最终测试与重复验证共用该配置。
- `scripts/check_data.py` 用于核对数据集形状、标签类别和像素范围；各模型脚本均使用固定的训练集/验证集划分随机种子 `random_state=23`。
- 测试集仅用于最终模型评价，不参与超参数搜索；报告中的实验数据应与上述脚本的最终运行结果保持一致。